

University of Southampton
Faculty of Engineering and Physical Sciences
School of Electronics and Computer Science

Yike Zhang, yz50u22

21st April 2025

Wearable Device for Badminton Motion Detection

Project supervisor: Doctor Alex Weddell

Examiner: Doctor Dimitra Georgiadou

A project report submitted for the
award of Beng Electrical and Electronic Engineering

University of Southampton

ABSTRACT

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES SCHOOL OF ELECTRONICS AND COMPUTER SCIENCE

By: Yike Zhang

Today, badminton motion analysis systems incorporating wearable sensors and machine learning models allow professional athletes to receive expert guidance more easily, while badminton motion analysis systems incorporating wearable sensors and machine learning models have enabled professional athletes to receive expert guidance more easily, existing systems often remain too costly, bulky, or technically complex for amateur users, limiting their accessibility and daily usability. Amateur players would benefit more from a smaller, more convenient, easier-to-understand, and more affordable wearable device. This project aims to design and implement a wearable device focused on real-time detection of badminton stroke movements, may facilitate improved access to expert-level guidance by replicating aspects of professional systems in a more accessible form. The device can collect motion and physiological data, which are transmitted to a mobile device for analysis via a Bluetooth Low Energy (BLE) module. The project encompasses hardware design, data acquisition, signal processing, and the development of machine learning-based action classification algorithms. The project has completed hardware coding, BLE communication initialisation, and filtering algorithm implementation. Beyond these technical completions, a functional prototype was successfully deployed and tested in real-world scenarios with badminton club participants. The action classification reached 77.37% accuracy for badminton strokes such as forehand, backhand, and smash. Additionally, the system was able to generate real-time, personalised coaching feedback through an integrated web interface using the ChatGPT API.

Statement of Originality

- I have read and understood the [ECS Academic Integrity](#) information and the University's [Academic Integrity Guidance for Students](#).
- I am aware that failure to act in accordance with the [Regulations Governing Academic Integrity](#) may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

I have acknowledged all sources, and identified any content taken from elsewhere.

I have acknowledged all sources in the report and clearly cited any content, figures, or ideas taken or adapted from external references.

I have used external resources.

The project uses the ArduinoBLE library by Arduino to implement Bluetooth Low Energy communication functions, and the SparkFun MAX3010x Sensor Library (version 1.0.5) for heart rate and SpO₂ signal acquisition and processing. Official Arduino tutorials for the Nano 33 BLE Sense Rev2 board and BLE device-to-device communication were referenced during sensor initialization and BLE service configuration. The OpenAI ChatGPT API (gpt-4o) was used to generate personalised coaching feedback based on recognised motion data. Additionally, Python libraries including pandas, scikit-learn, and joblib were used for machine learning model training, evaluation, and deployment, while the MATLAB BLE toolbox was employed for real-time data collection and CSV logging.

I did all the work myself, or with my allocated group, and have not helped anyone else.

I carried out all work myself, including hardware implementation, data acquisition, and software development, and did not help any other individual or group with their work.

The material in the report is genuine, and I have included all my data/code/designs.

The material in the report is genuine, and I have included all source code, raw and processed data, and hardware design information necessary to reproduce the results.

I have not submitted any part of this work for another assessment.

I confirm that no part of this work has been submitted for any other assessment, either within or outside the University

My work did not involve human participants, their cells, or animals; it only involved collecting motion data.

Approval was obtained from the University of Southampton Ethics and Research Governance committee (Reference Number: ERGO/FEPS/79094). All procedures were conducted following ethical guidelines, and informed consent was obtained from all participants.

Contents

List of Figures	6
List of Symbols	7
1 Introduction	8
1.1 Motivation	8
1.2 Project Objectives	8
1.3 Scope and Deliverables	9
2 Theoretical Background	9
2.1 Motion Detection Techniques for Sports	9
2.1.1 Existing Projects	9
2.1.2 Hardware Selections	10
2.2 Attitude Estimation Algorithms	13
2.3 Motion Analysis Methods	14
3 System Overview	15
3.1 Hardware Architecture	15
3.2 Software Architecture	17
3.3 Data Flow and System Workflow	18
4 System Implementation	20
4.1 Sensor Implementation	20
4.2 Mechanical Design	22
4.3 Machine Learning-Based Action Classification	24
5 System Testing, Performance, and Feedback	27
6 Project Management	35
6.1 Project Time Management	35
6.2 Project Risk Management	37

7 Conclusion and Future Work	38
7.1 Summary of Achievement	38
7.2 Limitations	39
7.3 Future Works	39
Bibliography	40
Appendix	43
A Formulas	43
B BLE UUID Table	43
C Sensor Specifications	44
D Ethics Approval Forms	45
E Code Examples	47

List of Figures

1	Example of Euler Angles reproduced from [11]	13
2	Overview of the Integrated Device	17
3	Software Architecture Overview	19
4	Code Implementation of I2C Scanner	20
5	Technical Drawing of Device Design	23
6	An earlier version of the design	24
7	When the device is initialised, the information printed by the Serial Monitor	28
8	Outputs on Mobile APP LightBlue	28
9	An example of Collected Data	29
10	Training Evaluation of Trained Model of Simple Motions	29
11	When the device operates properly	29
12	Training outcome of the initial Algorithm	30
13	Training outcome of Final Algorithm	31
14	Code of API Testing Program	32
15	API Testing Outcome	32
16	UI of Localhost Website	33
17	Example of Forehand	34
18	Initial Gantt Chart	36
19	Fianl Gantt Chart	37
20	Pin layout of Arduino Nano 33 BLE Sense Rev2, reproduced form [9]	44
21	System Diagram of MAX30102, reproduced form [22]	44
22	Signed SOP Form	45
23	Ethics Approval Submissions	45
24	Ethics Application Form	46
25	Code example of Reading Data	47
26	Code example of Establish BLE Connection	48
27	Code example of Heartrate and SpO_2 Algorithm	48
28	Peak Count Algorithm	49
29	The function for Connecting and Data Storage	49
30	Code example of Data Processing Progress in Pyhton	50

31	Code example of UI	51
32	Code example of Extended 16 Features	52

List of Symbols

IMU	Inertial Measurement Unit
Hz	Hertz (Unit of Frequency)
SpO_2	blood oxygen saturation
KNN	K-Nearest Neighbor Algorithm
SVM	Support vector machine
BLE	Bluetooth low energy
ϕ	Roll (Used in this report)
θ	Pitch (Used in this report)
ψ	Yaw (Used in this report)

1. Introduction

1.1. Motivation

Modern professional badminton players can often benefit from the guidance provided by expert coaches. With the integration of wearable sensors and machine learning technologies, advanced badminton motion analysis systems are increasingly being adopted for training and gameplay. However, for amateur badminton players, participating in periodic badminton courses offered by clubs or other organisations often comes with challenges, such as the time it takes to participate and the distances between home and clubs. Purchasing and using professional badminton analysis systems can be expensive or overly complicated. So, designing a wearable device that is cost-effective, portable, and user-friendly would enable users to effortlessly monitor their performance data and receive personalised recommendations, helping them improve their techniques. Furthermore, it enhances their enjoyment of the sport and reduces the risk of injuries during play.

1.2. Project Objectives

The core objective of this project is to develop a wearable device and miniaturised device based on a single sensor node for badminton stroke recognition, targeting amateur sports enthusiasts to enhance their real-time motion feedback and performance evaluation capabilities. The system aims to capture users' movement and physiological signals to recognize different badminton strokes and provide real-time personalised feedback to amateur players. It is designed to be lightweight, wearable, and capable of unobtrusive data acquisition during play. Captured data should be processed efficiently to classify stroke types and generate actionable improvement suggestions. The system must offer reliable operation, intuitive user interaction, and the ability to support continuous skill development without needing professional supervision. Initially, data was collected from experienced badminton players through self-conducted recording sessions to train a stroke classification model. Subsequently, when amateur badminton players use the device, their real-time motion data will be compared against the trained model. The comparison results will then be input into a large-scale artificial intelligence system to generate personalized recommendations, with both the collected data and the generated feedback visualized through platforms such as web interfaces.

1.3. Scope and Deliverables

This project covers hardware assembly, embedded development, data communication, web interface construction, data acquisition programming, and data structure design. The system components include the Arduino Nano 33 BLE Sense Rev2, the MAX30102 module, a 3D-printed enclosure, and wearable mounting structures. The development work will be limited to firmware-level sensor drivers, data acquisition programs, attitude estimation algorithms, BLE characteristic configuration, a front-end display interface based on Flask, and MATLAB or Python scripts for data reception and storage. The project deliverables include a functional prototype of the sensor hardware, a set of host-side scripts supporting BLE interaction, a collection of data samples for training classification models, and a basic web-based data visualisation interface. The data collected during the project will be limited to the three-axis motion data of the wrist. Physiological data collected by the MAX30102 will not be stored and will only serve as a reference for users. All data collected in this project will be anonymous, containing no identifiers that can link the data to individual participants, and the project has received ethical approval from the Ethics Committee of the University of Southampton.

2. Theoretical Background

2.1. Motion Detection Techniques for Sports

2.1.1 Existing Projects

In sports motion analysis, wearable IMU(Inertial Measurement Unit) sensors have become a prominent method for accurately identifying and classifying motion patterns. For instance, Camomilla et al. [1]. During their research, they used IMU, which includes an accelerometer, a gyroscope, and a Magnetic sensor, and analysed over 37 sports. They proved that the IMU can be used to identify and classify sports. A particularly relevant study by Cutti et al. (2008) proposed a calibration procedure for upper limb joint kinematics using wearable magnetic and inertial measurement units based on bony landmark identification [2]. Similarly, Lebel et al. (2018) applied wearable IMUs to running gait analysis, enhancing the understanding of biomechanics and the risk factors for injuries [3].

In badminton motion analysis, accurately identifying and classifying stroke actions is a critical research area. Wang et al. (2016) proposed a method for badminton stroke recognition based on Body Sensor Networks (BSNs) [4]. In their study, multiple IMU sensors were placed at key locations on the athlete's body, including the wrist, arm, and legs, to form a comprehensive network for capturing motion data. Wang et al. adopted a multi-sensor setup to obtain comprehensive motion data. While Camomilla et al. emphasized the broad applicability of IMUs across sports, their work lacked domain-specific optimizations, limiting direct transferability to badminton. Cutti et al.'s approach offered precise limb joint calibration, but required cumbersome wired systems unsuitable for mobile use. Lebel et al. focused on gait analysis, which differs structurally from upper limb dynamics in racket sports. In contrast, Wang et al.'s multi-sensor BSN system offered richer motion data but increased system complexity, cost, and wearing discomfort. It should also be noted that Wang et al. evaluated the recognition performance of individual sensor nodes. Their results showed that the wrist sensor node alone achieved the highest individual accuracy among the tested locations, reaching an average recognition accuracy of 81.63%. This suggests that, while a multi-node system enhances overall robustness, a well-placed single sensor (such as on the dominant wrist) can still deliver relatively high accuracy, particularly when paired with appropriate signal processing and feature extraction techniques.

2.1.2 Hardware Selections

In the study conducted by Cutti et al., a sensor named MT9B was employed, which comprised a 3D accelerometer, a 3D gyroscope, and a 3D magnetometer, operating at a fixed sampling rate of 100 Hz. This sensor required a wired connection to a computer and did not support wireless transmission. In the study by Lebel et al., Attitude and Heading Reference Systems (AHRS) were utilized to analyze motion performance, comparing systems such as the Xsens MTx, APDM Opal, and Inertial Labs OSv3.

In the research by Wang et al., each sensor node comprised a triaxial accelerometer (ADXL326), a single-axis gyroscope (LY550), and a biaxial gyroscope (LPR550). As these sensors lacked integrated Bluetooth functionality, ZigBee modules were implemented to configure the sensor nodes, and a wireless receiver module was connected to the computer for data acquisition.

In this project, the focus is on developing a miniaturized device based on a single sensor node; therefore, sensor systems similar to those used by Cutti et al. and Lebel et al. will not be considered. Instead, designs akin to that proposed by Wang et al., involving a microcontroller with integrated Bluetooth functionality combined with an additional IMU module, are considered more appropriate. Several candidate solutions are summarized in Table 1. Among the compared hardware options, professional-grade solutions like Xsens or Shimmer offer high accuracy and sampling rates, but their prohibitive cost and setup complexity render them impractical for casual or amateur users. Raspberry Pi and ESP32 platforms offer programmability and wireless communication but require additional IMU modules, increasing power consumption and form factor, which may hinder integration in a wearable form. Arduino Nano 33 BLE Sense Rev2, although limited in memory and requiring manual programming of BLE communication, provides a compact and integrated platform combining BLE and IMU in one unit. This balance of cost, size, and functionality makes it the most suitable choice for this project's lightweight, wrist-mounted wearable application.

Option	Advantages	Disadvantages	Cost (£)
1. Raspberry Pi 4B + External IMU Module[6]	• User-friendly • Supports MicroPython and C/C++ programming.	• Requires an external sensor module, increasing size and power consumption. • Requires an additional wireless module.	• £60+
2. ESP32 Series Development Board + External IMU Module[5]	• Built-in Wi-Fi and BLE. • Smaller size	• Also requires an external IMU module. • Higher power consumption; not conducive to portable sports equipment applications.	• £35+
3. Professional Sports Sensor Equipment (e.g., Xsens[7], Shimmer[8])	• High precision and high sampling rate; professional-level motion capture performance.	• Expensive. For example, it costs €430 to purchase a Shimmer IMU unit. • Complex equipment.	• £370+
4. Arduino Nano 33 BLE Sense rev2[9]	• High Integration, the BLE and IMU modules are integrated on board. • Low Power Consumption.	• 256 KB of SRAM and 1 MB of flash limit the size of programs. • Relatively harder to program: i.e. Algorithms and BLE need to be established by programming.	• £50

Table 1: Comparison of Hardware Options

2.2. Attitude Estimation Algorithms

Attitude estimation involves determining an object's orientation and rotational state in space. In three-dimensional space, Euler angles can describe the orientation of a rigid body that requires three independent angular parameters, which consist of roll (ϕ), pitch (θ), and yaw (ψ). Roll refers to the rotation around the X-axis and represents the left or right tilt of the object. Pitch refers to the rotation around the Y-axis and indicates the forward or backward tilt. Yaw refers to the rotation around the Z-axis and describes the direction of the object within the horizontal plane[10]. An example of Euler angles is shown in Fig.1.

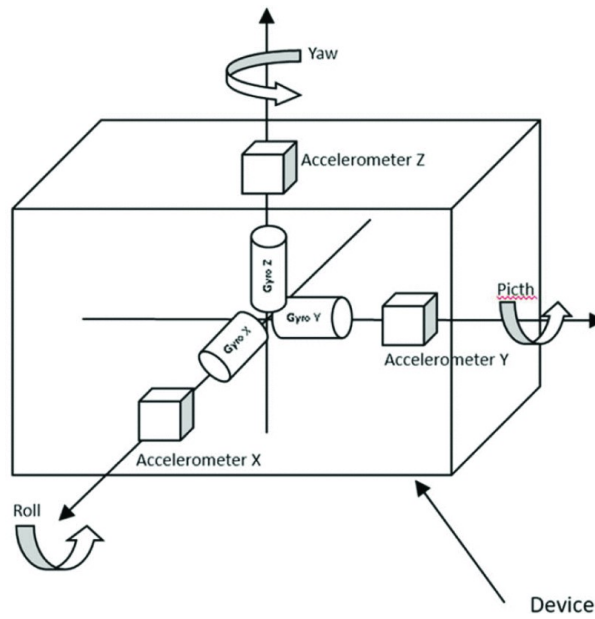


Figure 1: Example of Euler Angles reproduced from [11]

An accelerometer measures the gravitational components along each axis, from which the roll and pitch angles can be calculated through geometric projection. A magnetometer measures the horizontal components of the Earth's magnetic field and can be used to estimate the yaw angle (heading). A gyroscope measures the angular velocity around each axis, and the angular displacement can be obtained through integration. The detailed derivation and implementation of these calculations are provided in the Appendix, referencing the work of Sabatini (2006) on attitude estimation using inertial sensors [12] and the summary by Noureldin et al. on yaw angle estimation methods using magnetometers [13].

Each type of sensor within the IMU exhibits inherent limitations. For example, gyroscopes suffer from drift errors during long-term integration; accelerometers show poor stability during

dynamic movements; and magnetometers are vulnerable to disturbances from environmental magnetic fields. To address these issues, a complementary filter is adopted in this project to fuse data from the three sensors, leveraging the long-term stability of accelerometers and magnetometers and the short-term dynamic responsiveness of gyroscopes to generate smoother and more stable attitude angle estimations. The filter design is based on the nonlinear complementary filter structure proposed by Mahony et al. [14], with specific weighting factors and computational models detailed in the Appendix.

During data acquisition, the IMU is susceptible to external interferences, including involuntary human tremors while wearing, mechanical vibrations during motion, and electromagnetic interference (EMI) from the environment. Jankowski et al. conducted a systematic experimental study on the sources of vibration-induced noise in MEMS accelerometers and identified the inherent vibration of mechanical structures as one of the primary contributing factors [15]. Caballero et al. used electromagnetic tracking sensors to evaluate human movement variability and demonstrated that natural tremors can significantly impact sensor accuracy [16]. Pahl et al. validated the drift and nonlinear changes observed in gyroscopes and accelerometers when subjected to electromagnetic interference [17].

To suppress high-frequency disturbances, a Butterworth low-pass filter is applied during the data preprocessing stage. This filter features a flat frequency response within the passband and effectively attenuates high-frequency noise without significantly distorting the main components of the signal. The principles and characteristics of Butterworth filters were thoroughly analysed by Butterworth in his seminal 1930 paper [18].

2.3. Motion Analysis Methods

In the study of motion analysis methods, Ghosh et al. proposed a comprehensive motion modelling framework for table tennis stroke recognition. They developed a wearable sensor module system by attaching an IMU to the wrist to capture tri-axial acceleration and angular velocity signals. Statistical features such as mean, standard deviation, maximum, and minimum values were extracted from each signal segment using a sliding window approach [19]. These features were then input into a neural network model built using TensorFlow for classification training,

followed by quantisation using TensorFlow Lite to adapt the model to the target hardware platform. The motion analysis method adopted in this study constructed input vectors based on local statistical features of IMU signals and employed a shallow neural network for multi-class stroke action classification.

In existing studies on badminton motion recognition, machine learning methods have been widely applied for IMU sensor-based data modelling tasks. Anik et al. proposed using statistical variations of orientation angle sequences as primary features, combined with stroke duration and the position of the maximum acceleration point to construct feature vectors, utilising support vector machines (SVM) and k-nearest neighbours (k-NN) algorithms for action classification [20]. SVM offers good generalisation capabilities and handles small-sample high-dimensional spaces effectively. At the same time, k-NN is simple in structure and fast to implement, making it suitable for preliminary classification tasks under the real-time requirements of wearable devices.

In addition to static feature modelling, some studies have introduced temporal structure modelling methods. For instance, hidden Markov models (HMM) have been used to represent action sequences as state paths, estimating stage transition probabilities based on observed sequences, which is suitable for recognising actions with sparse labels or continuous stages [4]. Building upon this, deep learning models such as long short-term memory networks (LSTM) have been applied to modelling multi-phase swing processes in sports like badminton, leveraging their capability to learn temporal dependencies for posture sequence classification. Seong et al. employed an LSTM structure to process complete multi-phase stroke action sequences and incorporated a bidirectional recurrent structure to introduce contextual information and enhance classification accuracy [21].

3. System Overview

3.1. Hardware Architecture

The system is custom-encapsulated in the form of a wristwatch and is worn on the user's wrist. The decision to use the wrist as the sensor placement site is based on two considerations. First,

badminton stroke actions predominantly originate from wrist and arm movements, making the wrist an ideal location to capture dynamic motion characteristics. Second, prior studies such as Wang et al. (2016) reported that wrist-mounted sensors achieved the highest single-node recognition accuracy among various body positions. The internal structure integrates the main control board, sensors, battery, and charging management module. At the same time, the external casing is fabricated through lightweight 3D printing to ensure mechanical stability and user comfort during physical activities. The system's core control unit is the Arduino Nano 33 BLE Sense Rev2, featuring an nRF52840 Bluetooth chip that supports the BLE 5.0 communication protocol. It transmits attitude feature data streams to the host computer via custom-defined service and characteristic UUIDs. The Arduino board is integrated with the LSM9DS1 nine-axis inertial measurement unit (IMU), capable of real-time acquisition of tri-axial acceleration, angular velocity, and magnetic field data. A complementary filter is employed to compute the raw sensor outputs' roll, pitch, and yaw attitude angles. After filtering and differential processing of the sensor data, the firmware periodically transmits the results to the computer via BLE broadcasting, enabling real-time data acquisition and analysis.

The bottom of the system is equipped with a MAX30102 biosensor module for measuring the user's heart rate and blood oxygen saturation. This sensor utilises photoplethysmography (PPG) to output SpO_2 and pulse rate data, with direct skin contact to improve signal quality. It should be noted that the physiological parameters collected by the MAX30102 are not involved in subsequent motion recognition or machine learning model inputs; they are provided solely for users' reference to monitor their physical status after training sessions or competitions and are not utilised in system recognition logic or algorithmic inference.

Power is supplied by a 3.7v, 350mAh rechargeable LiPo battery. To ensure sufficient operating time for typical training sessions and data collection activities, the system was designed with a target battery life of at least 10 hours. Based on the measured average current consumption of approximately 25 mA by the sensors and main board during operation, a 3.7V, 350mAh rechargeable LiPo battery was selected. This configuration enables continuous operation for approximately 11.7 hours. The battery management module is based on the TP4056 chip, providing overvoltage, overcurrent, and thermal protection features, and supporting automatic USB power switching and safe charging. Overall, the hardware system offers strong portability

and energy efficiency, meeting the basic requirements for battery life, stability, and real-time data transmission in wearable sports monitoring applications. The figure2 shows an overview of the integrated device.

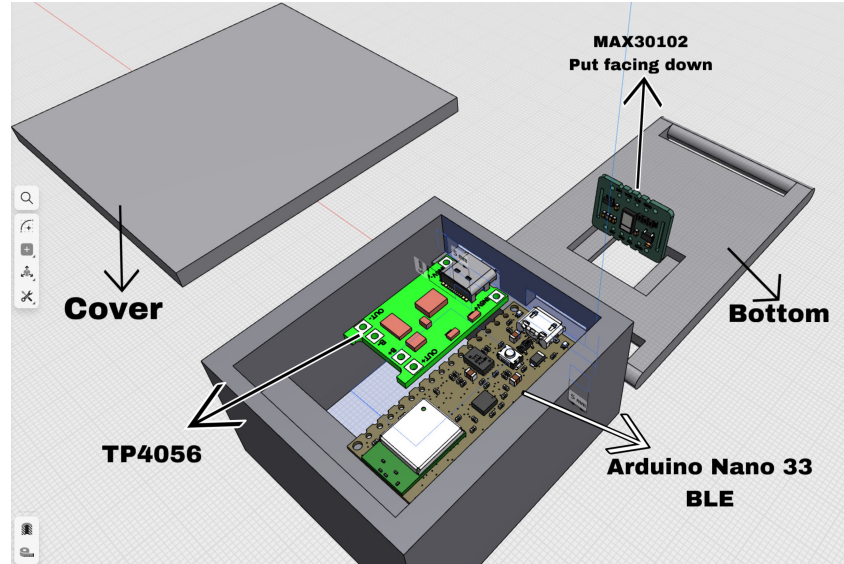


Figure 2: Overview of the Integrated Device

3.2. Software Architecture

The software system has two main components: the embedded control and host-side data analysis programs. The embedded program is developed in C++ based on the Arduino framework and is primarily responsible for sensor initialisation, continuous sampling, data filtering, differential computation, peak extraction, and BLE broadcasting.

The host-side program is implemented using a combination of MATLAB and Python. The system reads the broadcasted attitude angle data via Bluetooth in the MATLAB scripts. The retrieved data are processed within the script to compute angular velocity and angular acceleration, and the results are stored in CSV files with timestamp formatting. These files are subsequently used for machine learning model training.

The trained model is then applied for motion recognition, comparison, and recommendation generation, which is implemented through Python scripts. The Python scripts, based on the pandas and scikit-learn frameworks, are responsible for reading the data, preprocessing features, loading the trained models, and outputting classification results. A local visualisation

and feedback system is developed using Flask, enabling the visualisation of the collected data, model training results, and personalised recommendations through a web interface.

3.3. Data Flow and System Workflow

The system operates in two stages. The first stage is data collection, during which motion data from professional badminton players and coach volunteers performing different types of strokes are collected to serve as standard reference templates. During this stage, the IMU continuously reads tri-axial motion data and transmits it to the computer, where it is stored as CSV files. Machine learning model training is conducted based on this dataset. The training process is performed in Jupyter Notebook, and a support vector machine (SVM) is employed as the classification model for this project.

The second stage is motion recognition and feedback. When users wear the device and perform badminton strokes, their motion data is transmitted to the computer via BLE. A Python script receives the incoming data and applies the previously trained model to recognise which type of stroke the user has performed. The recognised action is then compared with the standard stroke templates stored in the model. The Python script is also integrated with the ChatGPT API, allowing the comparison results to be passed to ChatGPT for generating personalised feedback aimed at helping users improve their performance. An overview of software architecture is shown in Fig.3. The diagram illustrates the complete software architecture, showing the interaction between the embedded firmware, host-side programs, and external services. It distinguishes between two operational stages: MATLAB-based data collection and model training, and Python-based real-time inference and feedback. Official libraries were used in software design, and the self-written code was used to implement the functions.

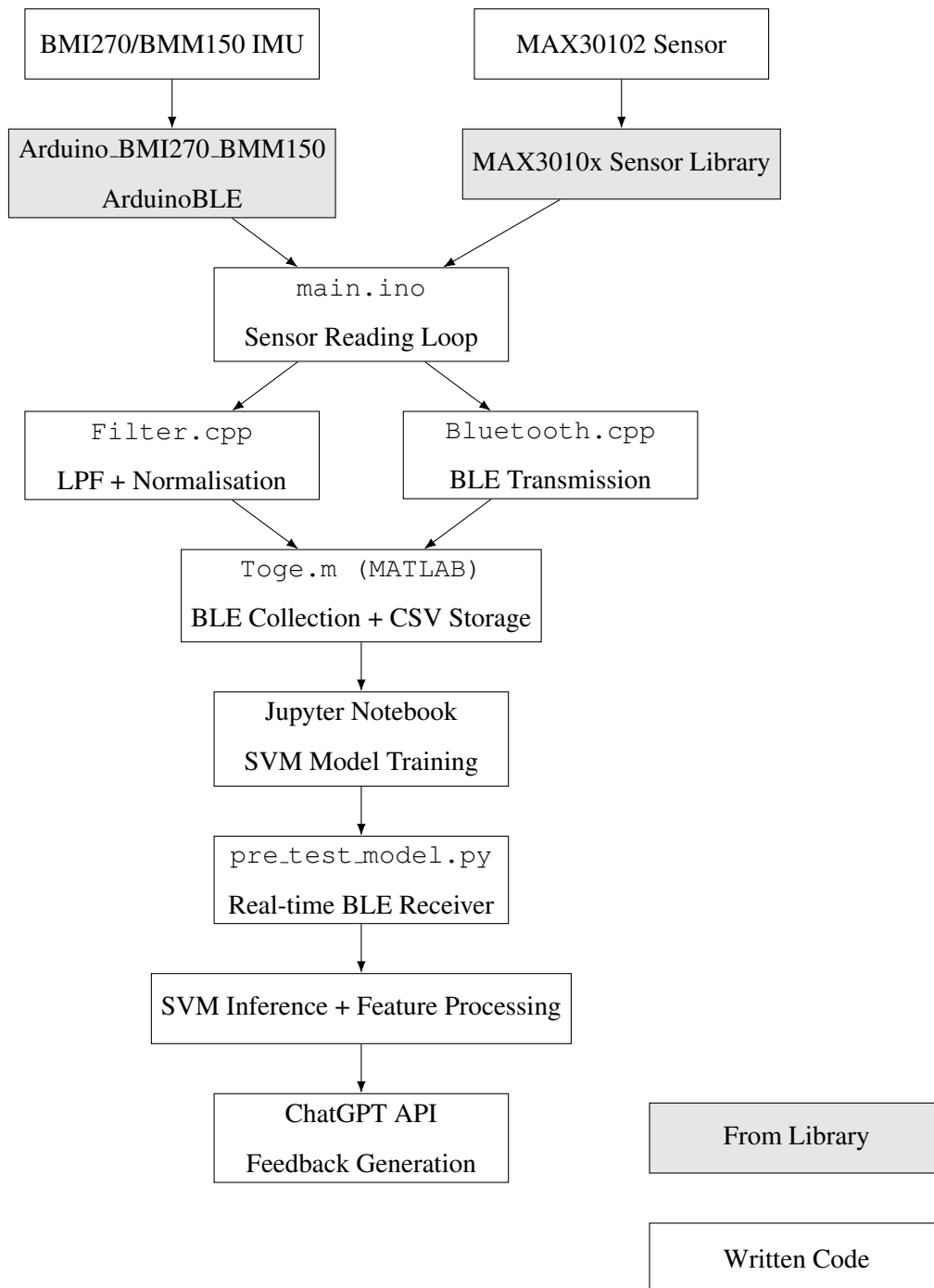


Figure 3: Software Architecture Overview

4. System Implementation

4.1. Seneor Implementation

The main control program completes serial port initialisation and device identification for the IMU and MAX30102 sensors within the setup() function. Using the I2C protocol, the system performs address scanning to verify peripheral connection status 4. During the sensor initialisation process, error detection mechanisms are incorporated. If a device fails to respond, an exception is reported via the serial port, and a timeout strategy is executed to ensure fault tolerance in the subsequent data acquisition workflow.

```
void i2cScanner() {
  Serial.println("Scanning for I2C devices...");
  byte count = 0;
  for (byte i = 1; i < 127; i++) {
    Wire.beginTransmission(i);
    if (Wire.endTransmission() == 0) {
      Serial.print("Found I2C Device at 0x");
      if (i < 16) Serial.print("0");
      Serial.print(i, HEX);
      Serial.println(" !");
      count++;
      delay(1);
    }
  }
  if (count == 0) {
    Serial.println("No I2C devices found.");
  } else {
    Serial.print("Found ");
    Serial.print(count);
    Serial.println(" device(s).");
  }
}
```

Figure 4: Code Implementation of I2C Scanner

The sensor hardware code is developed using official Arduino libraries[9]. The official Arduino tutorials[25] were consulted to learn how to use these libraries and their functions, but no tutorial code was directly reused. During inertial measurement data processing, the program acquires tri-axial raw data from the BMI270 accelerometer, gyroscope, and the BMM150 magnetometer. It then applies the imuLowPassFilter() function to perform exponential first-order low-pass filtering on the acceleration, angular velocity, and magnetic field intensity. Filtering parameters are defined as global variables, allowing adjustments to the filter coefficient to control the trade-off between response sensitivity and noise suppression. The filtered nine-axis data are subsequently fed into the attitude estimation algorithm, where roll, pitch, and yaw angles are individually computed. Roll and pitch are calculated using the arctangent function

based on acceleration components, while yaw is derived from the horizontal components of the magnetic field.

The program employs a complementary filter structure to fuse the gyroscope integration results with the sensor-derived angle estimates, independently updating the three attitude angles. The computed results are normalised and stored in global variables for use by the BLE interface. The angle computation process is controlled by timed sampling to maintain consistent integration steps, ensuring accurate conversion from angular velocity to angular displacement over real-time evolution.

Heart rate and blood oxygen saturation measurements are carried out using the MAX30102 sensor module[22]. The system configures the sensor in SpO_2 operating mode, with a sampling rate set to 100 Hz, an ADC range of 16384 nA. These parameters were selected based on the trade-off between signal quality and power consumption. A sampling rate of 100 Hz ensures sufficient temporal resolution to capture typical pulse waveforms without excessive computational load. The ADC range of 16384 nA provides adequate sensitivity for physiological signals while preventing saturation under moderate LED currents. The LED current was set to 60 (approx. 12 mA) to balance signal intensity and skin comfort, based on manufacturer recommendations and empirical testing on different skin tones. The program invokes the `update()` method from a custom `HeartRateSpO2` class during each main loop iteration to read red and infrared signals from the sensor and buffer them internally. Pulse wave envelope detection and signal decomposition algorithms are then executed to extract peak-to-peak intervals and light intensity variations per sampling cycle, from which heart rate (in BPM) and blood oxygen saturation (percentage) are calculated. The results are returned to the main program as floating-point values and assigned to global variables for BLE transmission and serial output.

Bluetooth communication functionality is implemented using the `ArduinoBLE` library[26]. A custom service UUID is configured during initialisation, and independent characteristic channels are registered for roll, pitch, yaw, heart rate, and SpO_2 measurements. After starting the BLE service, the device enters advertising mode, awaiting a central device connection. Upon successful connection, the program checks the BLE connection status in each main loop iteration; if connected, it invokes the `updateBLECharacteristics()` function to package and write

the latest sensor data to the respective characteristics, ensuring that connected devices receive motion and physiological signals at a near real-time frequency (approximately 100 Hz). The characteristics are transmitted in 32-bit floating-point format to ensure compatibility with mainstream BLE clients and low-latency decoding. To prevent resource conflicts, BLE data updates are prioritized over serial output, providing stable and continuous Bluetooth data transmission.

Serial debugging output is integrated into the main loop via the `printSensorData()` function. This function categorically prints the attitude angles and physiological parameters, allowing real-time monitoring of all metrics during the development phase. The output follows a fixed format, facilitating automatic parsing by serial reading programs and logging systems. Sampling interval control is applied during output to maintain synchronization with BLE updates, preventing data lag and buffer overflow issues.

4.2. Mechanical Design

The system's casing is designed to be a sports wrist-worn device, requiring compact volume, stable wearing, and a reasonable distribution of sensor exposure areas. The main body of the device measures 30×51×56 mm, with the longest point reaching 79 mm including the strap connection. These dimensions were determined primarily to accommodate all hardware components, while also taking into account ergonomic requirements during wrist movement. The device adopts a two-layer design: the upper layer houses the Arduino board and the charging module. Positioning the Arduino on the upper layer ensures that the physical thickness of the device does not obstruct Bluetooth signal transmission. Two USB openings are reserved: one for Arduino program uploading and the other for battery charging. These openings are symmetrically placed on the rear face of the upper shell, as shown in Fig. 5.

The lower layer serves as the battery compartment and accommodates the MAX30102 biosensor. A 17×14 mm opening is reserved at the bottom of the device to expose the sensor's photodiode and LED emitter window, enabling direct optical contact with the user's skin and ensuring reliable signal acquisition. This opening was dimensioned to match the active sensing region of the module and was refined through empirical testing. The casing is fabricated using 3D printing, and the full design schematic is provided in Fig. 5. The device is worn using a soft, adjustable wristband fixed through a hollowed-out side structure. The overall mass is approxi-

mately 30 grams, which avoids a significant load sensation during prolonged use.

The final casing design is the result of multiple iterations. An earlier prototype featured a similar layout but failed to account for the vertical clearance required by the Arduino board's pin headers, which made proper installation impossible. Additionally, the wristband attachment openings were initially too narrow to accommodate standard strap connectors. As shown in Fig.6, the upper side of the figure shows that the wristband attachment opening is designed to be 0.5 mm, which is ideally enough for a wristband's thickness, but didn't consider the 3D printing error. And the height of that design is 14.5 mm, which increased to 30 mm in the final design. The lower side of the figure showed that if Arduino was installed on the device, it would be higher than the coverage of the case. To resolve these issues, the casing dimensions were increased in the final version, providing sufficient height for internal components and ensuring compatibility with adjustable wristbands. Bluetooth signal performance and mechanical stability also improved as a result of the revised layout.

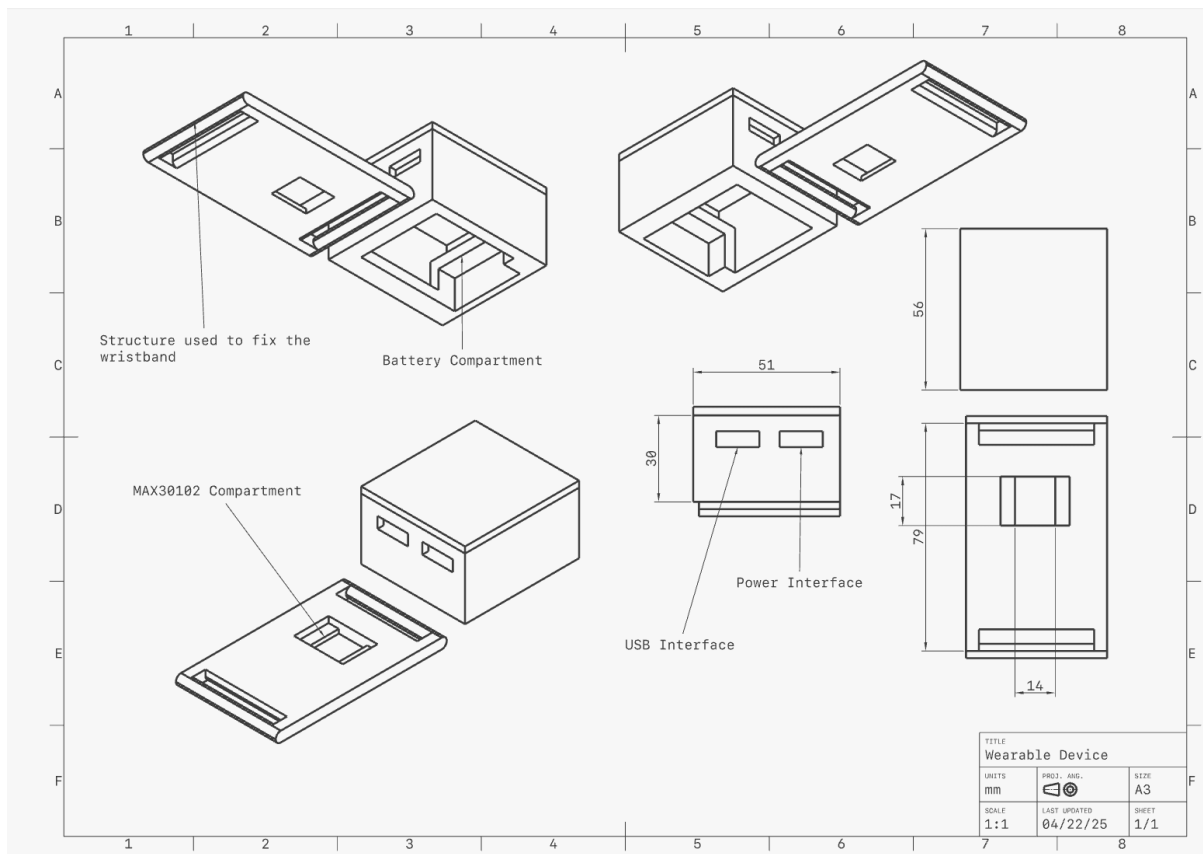


Figure 5: Technical Drawing of Device Design

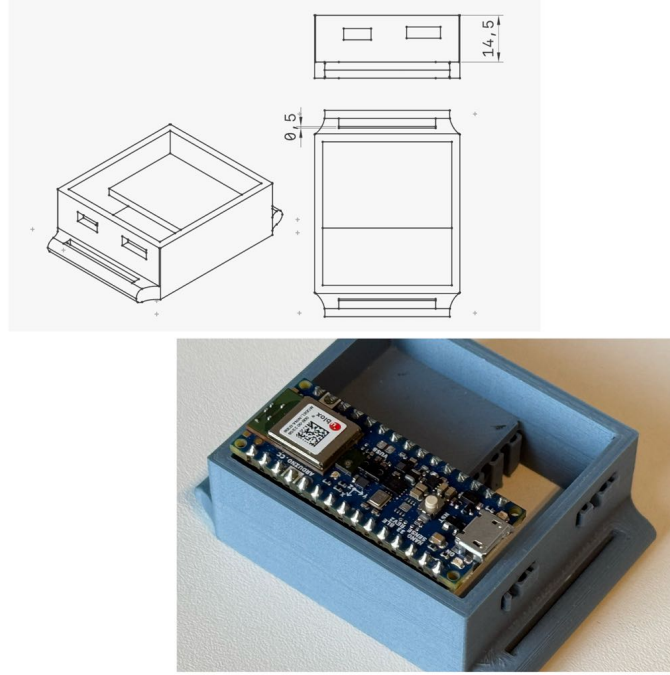


Figure 6: An earlier version of the design

4.3. Machine Learning-Based Action Classification

The training data were collected using the MATLAB script. The script searches for Bluetooth devices using `blelist` and connects to the target device. It continuously reads five raw features: roll, pitch, yaw, heartRate, and SpO_2 . To construct temporal features, the script calculates the angular velocities of roll, pitch, and yaw at each sampling instance by differencing the current and previous values, and further differencing yields angular accelerations. The angular velocity of the roll axis is buffered in a sliding array, and the number of extremum points is computed through second-order sign differencing within the window to obtain the peak count (`peak_count`). pseudocode 1 shows the peak count algorithm implemented in Matlab. These seven features are combined to form a feature vector, and after each sampling cycle, the vector, along with the timestamp and the five raw features, is saved into a CSV file in real time, the pseudocode 2 below shows the function connect to the Arduino and store the data into a csv file. Users manually append action labels as target variables for supervised learning.

Algorithm 1: Peak Counting Algorithm

Input: 1D array of numeric values *data*

Output: Number of peaks *peakCount*

```
1 if length of data  $\geq 3$  then
2   | peakCount  $\leftarrow 0$ ;
3 else
4   | Compute first-order differences:  $\Delta_1 \leftarrow \text{diff}(\text{data})$ ;
5   | Compute signs of  $\Delta_1$ :  $s \leftarrow \text{sign}(\Delta_1)$ ;
6   | Compute second-order difference:  $\Delta_2 \leftarrow \text{diff}(s)$ ;
7   | Count negative transitions in  $\Delta_2$ :
8   | peakCount  $\leftarrow$  number of elements where  $\Delta_2 < 0$ ;
9 end
```

Algorithm 2: BLE Connection and Data Logging Procedure

Input: Target device name

Output: CSV file with sensor data and derived features

```
1 Generate timestamped CSV filename;
2 Open file for writing, write CSV header;
3 Scan for BLE devices, locate target by name;
4 if target not found then
5   | Display error and exit;
6 Connect to device address with retries;
7 if connection fails then
8   | Display error and exit;
9 Bind characteristic UUIDs for roll, pitch, yaw, heart rate, SpO2;
10 Initialize buffers for time and data arrays;
11 while program not interrupted do
12   | Read raw sensor values from each characteristic;
13   | Compute velocities and accelerations via differencing;
14   | Count motion peaks from velocity buffer;
15   | Write timestamp and all features to CSV file;
16 end
```

The machine learning model training is conducted in Jupyter Notebook. The training script

uses pandas to load the collected and labeled CSV files. Labels are cleaned using `.str.strip()`, and effective action labels ('push', 'rest', 'swing', 'rolling') are selected. The input features include roll_velocity, pitch_velocity, yaw_velocity, roll_acceleration, pitch_acceleration, yaw_acceleration, and peak_count. Labels are encoded into integers using LabelEncoder. The dataset is split into training and testing sets with an 8:2 ratio, and features are normalized using StandardScaler. The model is initialised using `SVC(kernel='rbf')` and trained by calling `.fit()` on the training set. During testing, predictions are generated using `.predict()`, and performance metrics are reported using `classification_report()` and `accuracy_score()`. After training, the model (`pre_svm_model.pkl`), the scaler (`pre_scaler.pkl`), and the label encoder (`pre_label_encoder.pkl`) are saved to the local directory using `joblib.dump()`, for use by the inference script.

Support Vector Machine (SVM) was chosen as the final classification model based on the following considerations. First, the amount of data collected in this project is limited, and the number of motion samples is insufficient to support the data-intensive requirements of deep learning models such as LSTM or Transformer architectures. Among traditional shallow models, k-nearest neighbours (k-NN) offers simplicity and interpretability but requires traversing the entire training set during inference, making it unsuitable for real-time response demands in the web feedback system. Random Forest exhibits some robustness but tends to overfit and produce unstable prediction boundaries when feature dimensionality is low and class distribution is imbalanced. In contrast, SVM demonstrates significant advantages in small-sample, high-dimensional classification tasks, offering high post-training accuracy and short inference times, making it suitable for integration with web interfaces for real-time prediction.

The inference and feedback system is implemented through the Python script `pre_test_model.py`. The script uses `BleakScanner.discover()` to automatically scan and connect to the Bluetooth device. After establishing the GATT service, it reads the real-time data corresponding to each feature UUID. In each sampling cycle, the script computes angular velocities between two consecutive frames of roll, pitch, and yaw, calculates angular accelerations based on velocity differences, and performs extremum point counting using a sliding window over the roll_velocity signal to obtain the peak_count. The constructed feature vector is normalized using the pre-loaded scaler, following the same feature order as during training, and then fed into the pre-

trained SVM model for action prediction. The predicted label is decoded back into text using the label encoder.

After prediction, the system invokes `openai.ChatCompletion.create()`, passing the recognized action category as a prompt to the GPT-4o API. As I've mentioned in chapter 3.3, note that no raw sensor data or personal information is sent to the GPT API; only the predicted class label is included in the prompt. The model is requested to generate technical evaluations and personalised advice in the style of a professional coach. The generated suggestion, current action, and real-time data are packaged into a dictionary object and output as JSON data via the Flask endpoint `sensor_data`.

The front-end visualization is built based on the Flask framework. The web page template uses the HTML file, rendered via `render_template()` and loaded as the homepage under the route. The browser periodically requests the `sensor_data` endpoint to retrieve real-time JSON data and dynamically displays the page's current attitude derivatives, inferred action label, and textual advice. Upon system startup, the web service listens on local port 5000, allowing users to access `localhost:5000` via a browser to view the recognised stroke type and corresponding improvement suggestions.

5. System Testing, Performance, and Feedback

The program was first uploaded to the Arduino board during the initial testing phase and powered through a USB connection to the computer. Upon powering on, the device status could be monitored via the Serial Monitor in the Arduino IDE. If the system operated correctly, the Serial Monitor would display readings as shown in the Fig.7. At this point, Bluetooth initialisation would be complete, and the device would start broadcasting to the surrounding area. Using the mobile application LightBlue, the device could be successfully connected, and various sensor parameters could be observed, verifying that device initialisation, Bluetooth communication, and basic data reading were functioning correctly⁸. However, it should be noted that at this stage, it was not yet possible to conclude whether the sensor readings accurately reflected real motion states, requiring further validation.

```

Serial communication initialized
IMU initialized
Scanning for I2C devices...
Found I2C Device at 0x57 !
Found 1 device(s).
MAX30102 initialized!
BLE initialization succeed
BLE established, waiting for connection...

```

Figure 7: When the device is initialised, the information printed by the Serial Monitor

Services		Service UUID
		19B10000-E8F2-537E-4F6C-D104768A1214
0x19B10000-E8F2-537E-4F6C-D104768A1214		Read/Notified values
0x19B10001-E8F2-537E-4F6C-D104768A1214		Read Again
0x19B10002-E8F2-537E-4F6C-D104768A1214		Subscribe
0x19B10003-E8F2-537E-4F6C-D104768A1214		Cloud Connect
0x19B10004-E8F2-537E-4F6C-D104768A1214		OFF
0x19B10005-E8F2-537E-4F6C-D104768A1214		

Figure 8: Outputs on Mobile APP Light-Blue

The next step was integration testing. The device was mounted onto a breadboard via pins, and the MAX30102 sensor was connected to the Arduino's I²C interface (A4, A5) and power interface (3V3, GND) via wires. The device connection status was first verified using the pre-configured I²C Scanner functionality in the firmware. If operating normally, the Serial Monitor displayed outputs as shown, and the LED on the MAX30102 module lit up. After confirming basic hardware functionality, preparation was made to connect the battery for testing. The project used a 350mAh LiPo battery, with a fully charged voltage of approximately 4.2V, dropping to around 3.2V when near depletion. By measuring the battery voltage to ensure it was within the usable range, the battery was connected to the TP4056 input, and the TP4056 output was wired to the Arduino power input pin, providing additional stability protection through the TP4056's overvoltage, overcurrent, and thermal safeguards.

After these preliminary tests, initial accuracy testing was conducted by connecting the Bluetooth device via a custom MATLAB script. For this step, the tester held the device and performed three types of motions: waving, arm rotation, and front-back swinging. Each action was repeated across multiple sets, with each set lasting approximately two minutes. Labs were manually appended after collecting sufficient motion data, and a labelled dataset was created as shown in the Fig.9. This data was then used to train an initial model. The evaluation results of the trained model¹⁰ showed an overall accuracy of 88.95% and a macro-averaged F1-score of 0.90, indicating good classification performance. Among the categories, the rolling action achieved the best classification results, with a Precision of 1.00 and an F1-score of 0.98, exhibiting almost no misclassifications. The push and rest actions also demonstrated high accuracy

and recall. However, the swing action exhibited a relatively lower Recall of 0.72, suggesting that some swing samples were misclassified, indicating a need for further optimisation of feature extraction or class balance. Overall, the model showed stable and strong recognition capabilities across most action classes, making it a solid foundation for subsequent motion recognition systems.

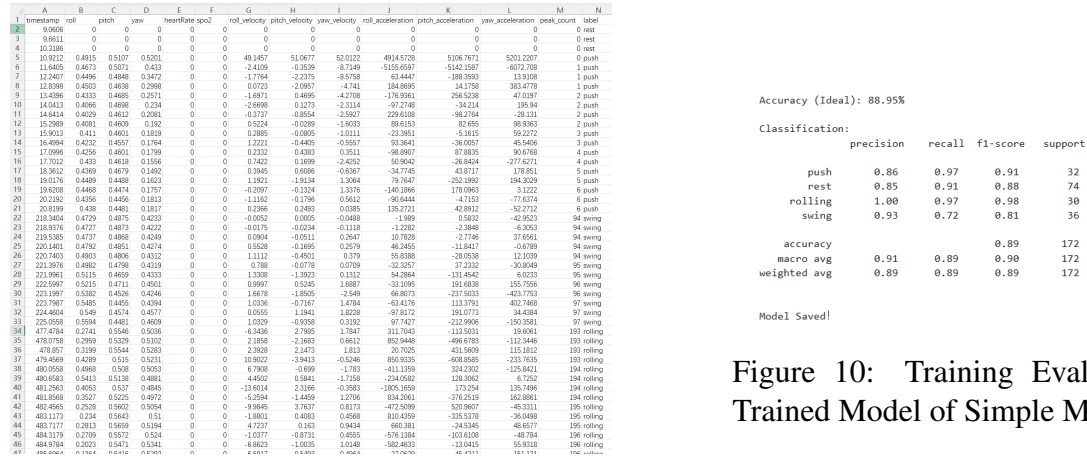


Figure 10: Training Evaluation of Trained Model of Simple Motions

Figure 9: An example of Collected Data

Before testing the fully assembled system, a standalone debug program, debug.py, was developed to ensure Python could successfully establish Bluetooth communication and display data on the Flask web interface. Running independently of the server framework, this script used the Bleak library to connect to the device and read each Bluetooth characteristic value sequentially. After each read, the script displayed both the floating-point value of each characteristic and the underlying raw byte stream, facilitating detection of data parsing errors or anomalies. During actual debugging, it was observed that data updates refreshed every 0.5 seconds, and the vast majority of data was correctly parsed. Occasionally, when the sensor experienced a momentary disconnection or Bluetooth interference, the script accurately captured and output error messages. As shown in the Fig.11, the device can operate normally.

```
F:\modelTrain>python debug.py
Scanning for BLE devices...
Found device: TOGENASHI_TOGEARI (89:40:91:1F:07:9C)
Connected to TOGENASHI_TOGEARI

--- Reading BLE Data ---
roll: bytes=[43, 205, 3, 63] float=0.5148493647575378
pitch: bytes=[186, 185, 243, 62] float=0.47602635622024536
yaw: bytes=[199, 21, 236, 62] float=0.4611036479473114
heartRate: bytes=[0, 0, 0, 0] float=0.0
spo2: bytes=[0, 0, 0, 0] float=0.0
```

Figure 11: When the device operates properly

After the completion of device assembly, collaboration was established with the University of Southampton Badminton Club (RecBad) to conduct device testing and data collection, with the assistance of two club members. After the participants wore the device and connected it via Bluetooth to a computer, they performed stroke actions. Four types of strokes were tested: Forehand, Backhand, Forehand Slice, and Smash. Multiple sets of each stroke type were recorded, with each set lasting approximately 2–5 minutes. Breaks were provided when participants experienced fatigue. After collecting sufficient data, a badminton motion detection model was trained using the acquired badminton stroke dataset.

The initial classification performance evaluation results are shown in the Fig.13 following the preliminary training of the model using the badminton stroke data. The support vector machine (SVM) model, constructed based on basic features such as angular velocity, angular acceleration, and peak count, exhibited relatively poor recognition performance. The test set accuracy was only 53.91%, and the macro-averaged F1-score was 0.50. Specifically, although the Backhand action showed a relatively high recall, its precision was low, indicating many misclassifications. While the Forehand action exhibited relatively good precision, it suffered from a high rate of missed detections. Recognition performance for the Forehand Slice and Smash actions was generally low, particularly for the Smash action, where the F1-score was only 0.34. These results indicate that the initially selected feature set was insufficient to adequately represent the dynamic characteristics of different stroke types, resulting in blurred classification boundaries and limited model performance.

Accuracy (Ideal): 53.91%				
Classification:				
	precision	recall	f1-score	support
Backhand	0.59	0.84	0.70	75
Forehand	0.70	0.29	0.41	72
Forehand Slice	0.52	0.59	0.55	56
Smash	0.33	0.35	0.34	40
accuracy			0.54	243
macro avg	0.53	0.52	0.50	243
weighted avg	0.56	0.54	0.52	243
Model Saved!				

Figure 12: Training outcome of the initial Algorithm

To address this issue, a systematic feature expansion was conducted. In addition to retaining the original features, information regarding the absolute spatial position of actions was intro-

duced. Furthermore, the absolute values of angular velocity and angular acceleration along each axis were calculated to enhance sensitivity to motion amplitude variations and reduce the directional noise’s impact. After the feature expansion, the input dimensionality increased from 10 to 16, effectively enriching the motion feature space. After the improvements, the retrained SVM model exhibited a significant performance enhancement. On the test set, the overall accuracy increased to 77.37%, and the macro-averaged F1-score improved to 0.76. Recognition performance across different action classes also became more balanced, with the Backhand action achieving an F1-score of 0.84, Forehand at 0.75, Forehand Slice at 0.79, and Smash at 0.64. As discussed in the background section, previous systems employing multiple IMUs or multi-sensor arrays have typically achieved classification accuracies of 85–90%. However, such systems rely on complex multi-point deployments and are less constrained by portability. In contrast, this system uses only a single wrist-mounted IMU under real-time and ergonomic limitations. For a four-class task, random guessing yields 25% accuracy, and simple heuristics, such as threshold-based peak detection, fixed value segmentation, or manually defined acceleration angle rules, these classification relies on manually set cutoffs rather than learning discriminative features. These approaches rarely exceed 50–60%. Therefore, under the current hardware constraints, the achieved 77.37% accuracy is acceptable.

```

Accuracy (Improved): 77.37%

Classification:

```

	precision	recall	f1-score	support
Backhand	0.80	0.89	0.84	75
Forehand	0.90	0.64	0.75	72
Forehand Slice	0.71	0.89	0.79	56
Smash	0.66	0.62	0.64	40
accuracy			0.77	243
macro avg	0.77	0.76	0.76	243
weighted avg	0.79	0.77	0.77	243

```

Model Saved!

```

Figure 13: Training outcome of Final Algorithm

To ensure that the system could reliably generate motion analysis and personalized suggestions, an independent test script, `api_testing.py`, was developed¹⁴. This script directly calls the OpenAI ChatGPT API to verify the API key’s validity and the server connection’s stability. By sending fixed test prompts and reading the returned messages, the API endpoint was confirmed to be responsive and capable of stably generating textual feedback. The successful API call

results are shown in the Fig.15.

```
F: > modelTrain > api_testing.py > ...
1 import openai
2
3 openai.api_key = "my api key"
4
5 try:
6     response = openai.ChatCompletion.create(
7         model="gpt-4o",
8         messages=[{"role": "system", "content": "You're an assistant"},
9                   {"role": "user", "content": "Test if OpenAI API is available"}]
10    )
11    print(response["choices"][0]["message"]["content"])
12 except Exception as e:
13     print(f"API request failed: {e}")
14
```

Figure 14: Code of API Testing Program

```

1. To test if the OpenAI API is available, you can make a simple request to the API endpoint. Here's a general approach you could take using Python and the 'requests' library:
2. **Install the Requests Library**: If you haven't already, install the 'requests' library using pip.
3. **Back**:
4. pip install requests
5.
6. **Write a Test Script**: You can use a small script to send a request to the OpenAI API. Ensure you have your API key available.
7.
8. python
9. import requests
10. # Replace 'your-api-key' with your actual OpenAI API key.
11. api_key = 'your-api-key'
12. headers = {
13.     'Authorization': f'Bearer {api_key}',
14.     'Content-Type': 'application/json',
15. }
16. # Example endpoint, such as for the ChatGPT model
17. url = 'https://api.openai.com/v1/chat/completions'
18. data = {
19.     'model': 'gpt-3.5-turbo',
20.     'messages': [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "Hello, world!"}]
21. }
22. try:
23.     response = requests.post(url, headers=headers, json=data)
24.     response.raise_for_status() # Raise an error for bad HTTP status codes
25.     print("API is available.")
26.     print(response.json())
27. except requests.exceptions.HTTPError as err:
28.     print("HTTP error occurred:", err)
29. except Exception as err:
30.     print("Other error occurred:", err)
31. ...
32.
33. **Run the Script**: Execute the script in your Python environment. If successful, you should receive a response from OpenAI indicating the API is available. The script will also print out part of the API response, such as model outputs.
34.
35. **Handling Exceptions**: The script will print specific error messages if there is an issue connecting to the API or an HTTP error response is returned, indicating the nature of any problems.
36.
37. This simple test should confirm whether the OpenAI API is accessible with your credentials.

```

Figure 15: API Testing Outcome

After completing system integration, a full round of testing based on real motion data was conducted using custom Python scripts. A participant wore the device on the wrist, which was connected to the computer via Bluetooth. The system continuously recorded the changes in roll, pitch, and yaw angles during movement, along with corresponding angular velocities, angular accelerations, absolute angular velocities, absolute accelerations, and peak counts, resulting in a total of sixteen sensor features. The data were uploaded in real time at an approximate frequency of once every 0.5 seconds and were simultaneously refreshed on the web interface. By accessing <http://localhost:5000/>, as shown in Fig.16, the webpage displayed two main sections: Sensor Data and Prediction. The Sensor Data module listed the real-time changes of each feature in detail. The interface responded smoothly, and the variations of all indicators were consistent and continuous.

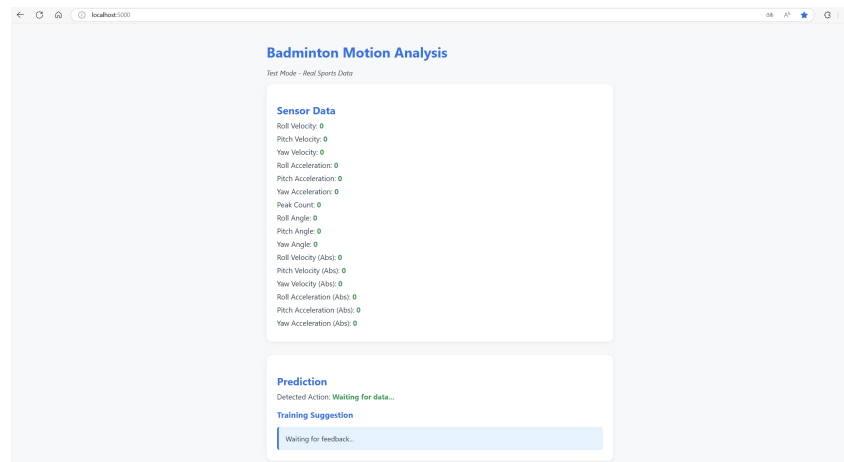


Figure 16: UI of Localhost Website

As shown in Fig.17, taking the Forehand action as an example, the system could classify the action shortly after completion and generate detailed, personalised training advice. During the stance phase, players should position their feet shoulder-width apart, with the front foot slightly forward and the rear foot turned outward, shifting the centre of gravity onto the ball of the front foot. The spine should be naturally extended with a forward lean of 15° – 20° , and the shoulders should remain relaxed and lowered. In the preparation phase, the racket arm's elbow should be slightly above shoulder level, forming a 90° elbow angle, with the racket head positioned behind the hand and pointing upwards to establish a loaded posture. To correct issues such as insufficient backswing or low elbow position, mirror practice or video playback was recommended. During the execution phase, players should drive the arm forward naturally from the shoulder, adding a controlled wrist snap at impact for power generation. The contact point should be 30–45 cm in front of and slightly to the right of the body, with a vertical racket face for accurate direction; if too close to the body, footwork drills were suggested. In the follow-through phase, the racket head should extend naturally forward and upward, with the elbow opening slightly outward and the racket head kept above the wrist to maintain motion continuity. Additional advice included using simulated targets to regulate the swing trajectory and timers to build rhythm, ensuring smooth transitions across all phases. Daily practice routines such as 20 sets of Shadow Drill swings (each over 60 seconds) combined with fixed-position Multi-feed drills were also recommended to improve consistency.

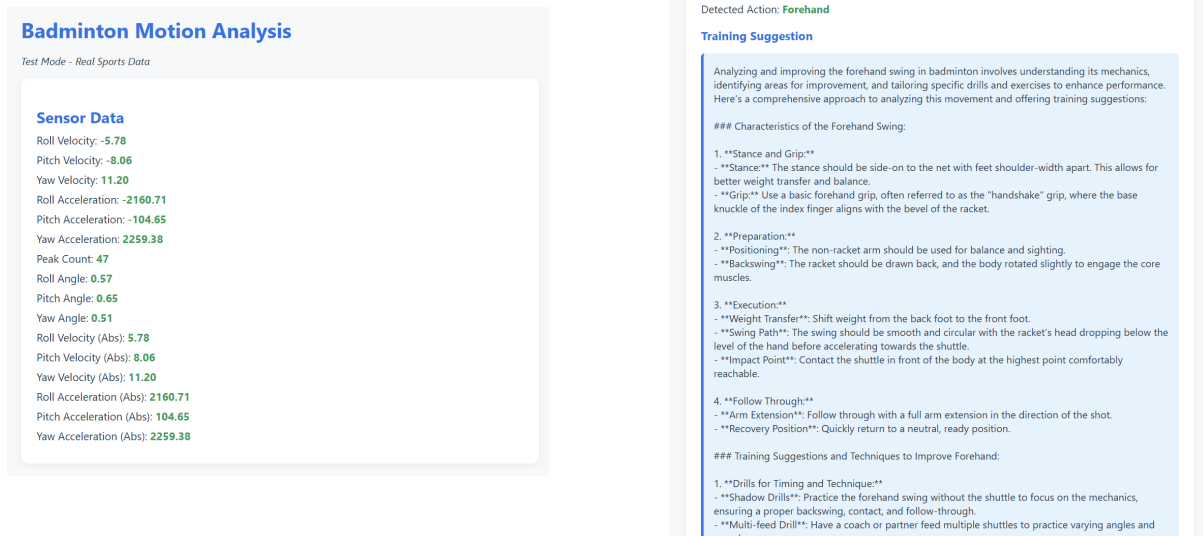


Figure 17: Example of Forehand

Participants also provided concrete suggestions for improving the device's wearing experience during testing. Most participants reported that the bottom structure of the device was insufficiently contoured to match the natural curvature of the human forearm. The current design did not fully account for the physiological arch formed by the natural bend of the forearm, leading to incomplete contact between the device and the skin. During large-amplitude movements such as swinging and body rotation, localised pressure increased, particularly on the wrist's dorsal side and the forearm's proximal region, causing discomfort. Additionally, some participants pointed out that the device's overall weight was relatively high. Although this did not significantly affect short-term use, during prolonged multi-session exercise tests, the accumulated weight increased the load on the arm muscles, leading to the early onset of fatigue, especially after performing high-frequency swinging actions. Some participants noted that compared to training without additional weight, wearing the current device version made the forearm muscles more prone to early fatigue and reduced the fluidity of movements towards the later stages of the testing session. Based on this feedback, participants generally recommended that future versions should optimise the curvature of the device's contact surface to better conform to ergonomic curves of the arm, and further reduce the overall device weight to minimise interference with natural motion patterns, thereby improving comfort and naturalness during prolonged wearing.

6. Project Management

6.1. Project Time Management

At the beginning of the project, a Gantt Chart was drawn as shown in the Fig18. Before the project commenced, based on the project objectives, the overall development process was broken down into specific subtasks, arranged sequentially according to their completion order, with the required duration for each subtask clearly estimated. The work was initially divided into six major phases: hardware development, embedded programming, data collection, machine learning training, system integration, final testing, and report writing. Within each phase, detailed tasks were further specified. In the initial estimation, it was assumed that software development would consume more time. However, once the hardware programming was completed, data collection could be initiated simultaneously with the development of the machine learning models and the Flask-based local server, thus saving time through parallelisation.

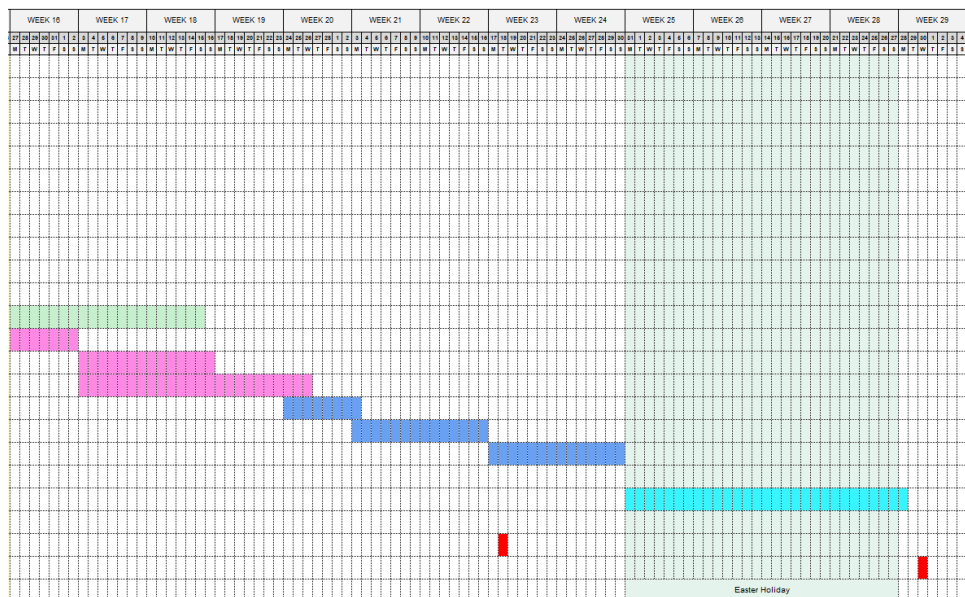
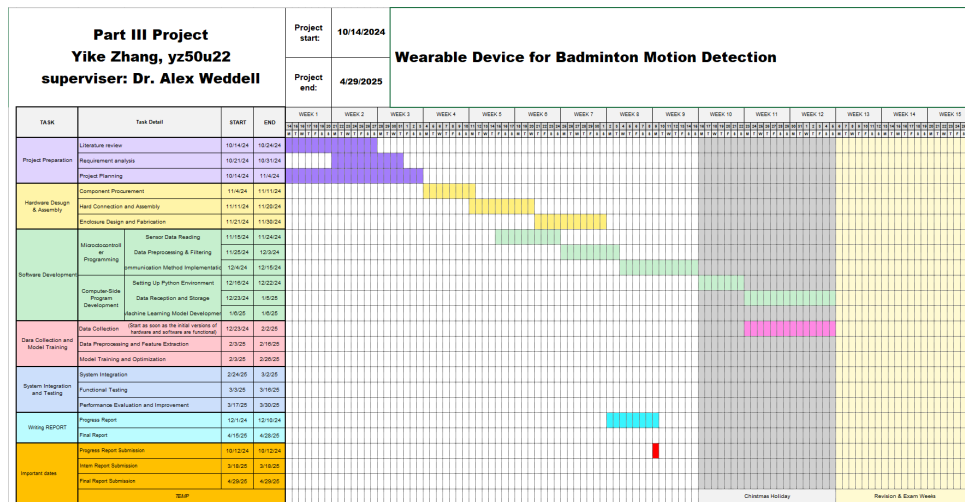


Figure 18: Initial Gantt Chart

Nevertheless, during actual implementation, difficulties encountered during the development of the Bluetooth communication module delayed data acquisition, as the inability to correctly read sensor data prevented the start of data collection. This resulted in a postponement of the hardware testing phase. Furthermore, toward the end of the project, an additional delay occurred when the Arduino board was damaged due to a short circuit. As no backup hardware was prepared, integration testing and further data collection were also delayed. Throughout the project execution, it was also observed that some time estimations made during the planning phase were inaccurate. For instance, data collection and system testing were found to proceed simultaneously rather than sequentially, as initially planned. Based on the actual project

progress and outcomes, an updated Final Gantt Chart was produced, as shown in the Fig.19.

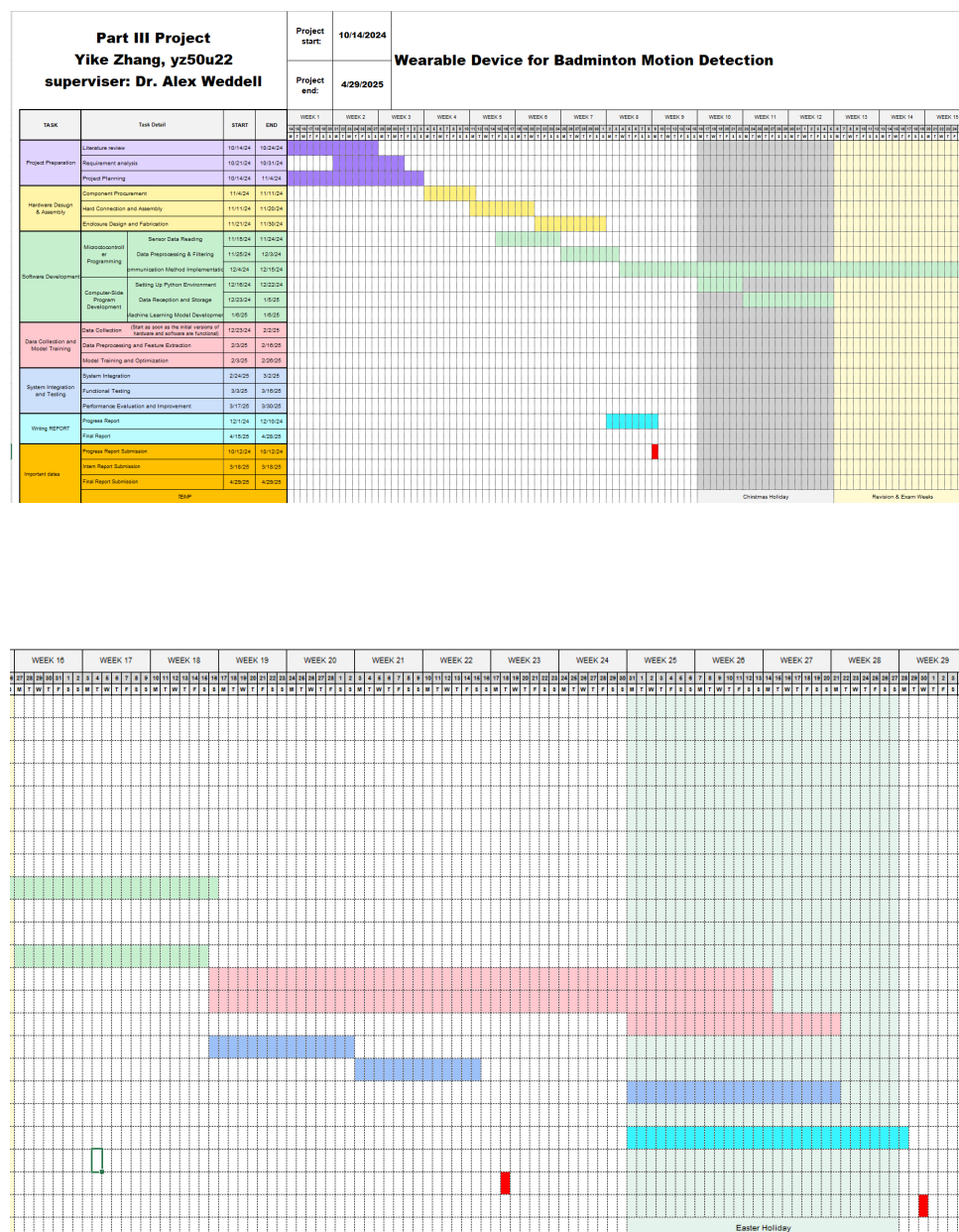


Figure 19: Final Gantt Chart

6.2. Project Risk Management

Since this project involves designing a wearable sensor and collecting data from participants, ethical approval from the University of Southampton Ethics Committee was required. This project's data are strictly limited to arm motion data, heart rate, and blood oxygen saturation. No personally identifiable information (i.e., data that can be linked to an individual's identity) will be collected, and all data will remain anonymous. Although participants may be asked

to complete a Consent Form, the consent forms will not be associated with the collected data. Additionally, participants retain the right to withdraw from the study at any time without consequence. Copies of the Ethics Application Form and the Consent Form are included in the Appendix of this report.

7. Conclusion and Future Work

7.1. Summary of Achievement

This project successfully developed a wearable badminton motion detection device based on the Arduino Nano 33 BLE Sense Rev2 platform. The system can reliably collect IMU data along with heart rate and blood oxygen saturation data. After applying filtering processes, standardized attitude angles are extracted. Using the BLE (Bluetooth Low Energy) module, both motion and physiological data are transmitted in real-time to mobile devices. Furthermore, through machine learning model training, the system is able to provide users with personalized feedback and suggestions.

During testing, the participant completed the four types of badminton strokes included in the experimental sample set without a predefined action sequence. During the motion, the roll-direction angular acceleration rapidly increased to nearly +2000 within a short period, while the yaw-direction acceleration exhibited peak values exceeding +2200. The roll angular velocity reached a high peak of approximately +15 at the initial stage of the action and then quickly reversed direction, generating a typical dynamic signature characteristic of stroke actions. Simultaneously, the web interface recorded a significant increase in the peak count value during the action cycle, reaching between 45 and 50, indicating that the rapid directional changes during the motion were effectively captured. The roll, pitch, and yaw angle data exhibited a continuous and smooth variation trend throughout the action, without sudden spikes or data loss, demonstrating the stability of the system's data acquisition chain.

Upon completion of each action, the system performed motion recognition based on the sixteen real-time collected features. It updated the detected action category in the Prediction section of the web interface. During testing, the system successfully recognised actions using the real-time sensor data and promptly displayed the updated recognition results on the web interface.

7.2. Limitations

Reflecting on the project, several limitations were identified. During testing, heart rate detection exhibited inevitable fluctuations and inaccuracies, particularly during intense physical activities, where signal stability declined, resulting in abnormally low or discontinuous heart rate readings. Blood oxygen saturation measurements performed well under resting conditions but showed reduced reliability during active movement. Using a single sensor node to monitor motion also presents inherent limitations. Although most actions could be recognised, in practical scenarios, minor differences between specific actions, or action deformations caused by factors such as fatigue, introduced inaccuracies into the data, leading to potential misclassifications. Additionally, although the action recognition model performed well on the collected dataset, the sample size remains relatively small. The clarity of the decision boundaries between action classes is limited, introducing a risk to the model's generalisation capability. While the web interface has been functionally completed, systematic optimisation for data synchronisation and page response efficiency under high-concurrency conditions has not yet been conducted.

7.3. Future Works

Further research on badminton motion detection and personalised feedback systems will be pursued in the future. It is intended to explore the use of multi-sensor systems by integrating lightweight, miniaturised sensors at key joint locations, connected to the main device through IoT technologies, to enhance motion data accuracy. Advanced algorithms will be introduced into sensor data processing to reduce attitude drift errors further. The motion recognition module will be expanded by increasing the dataset size, collecting samples from users of various skill levels and a wider variety of detailed action types to enhance model robustness and generalisation performance.

The web interface will be enhanced on the software side by adding modules for real-time training feedback, historical record management, and motion trend analysis. Development of a dedicated mobile application may also be considered. Future exploration could include multi-modal sensor fusion (e.g., combining electromyography, EMG, signals) to recognise more complex motion states. Further optimisation of device power consumption and battery life will also be pursued to support long-term monitoring during extended training sessions.

Bibliography

1. Camomilla, V., Bergamini, E., Fantozzi, S., & Vannozzi, G. (2018). Trends supporting the in-field use of wearable inertial sensors for sport performance evaluation: A systematic review. *Sensors*, 18(3), 873.
2. Cutti, A. G., Giovanardi, A., Rocchi, L., Davalli, A., & Sacchetti, R. (2008). Upper limb joint kinematics using wearable magnetic and inertial measurement units: An anatomical calibration procedure based on bony landmark identification. *Medical & Biological Engineering & Computing*, 46(2), 169–178.
3. Lebel, K., Boissy, P., & Hamel, M. (2018). Inertial measures of motion for clinical biomechanics: Comparative assessment of accuracy under controlled conditions—Current state and perspectives. *Sensors*, 18(2), 609.
4. Wang, Z., Guo, M., & Zhao, C. (2016). "Badminton Stroke Recognition Based on Body Sensor Networks," *IEEE Transactions on Human-Machine Systems*, vol. 46, no. 5, pp. 769–775, Oct. 2016, doi: 10.1109/THMS.2016.2571265.
5. Espressif Systems. Development boards. Retrieved from <https://www.espressif.com/en/products/devkits> [Accessed: 1 Dec 2024].
6. Raspberry Pi Foundation. Raspberry Pi 4 Model B specifications. Retrieved from <https://www.raspberrypi.com/products/raspberry-pi-4-model-b> [Accessed: 1 Dec 2024].
7. Movella. Xsens sensor modules. Retrieved from <https://www.movella.com/products/sensor-modules> [Accessed: 7 Dec 2024].
8. Shimmer Sensing. Wearable sensor products: Individual sensors. Retrieved from <https://shimmersensing.com/wearable-sensor-products> [Accessed: 1 Dec 2024].
9. Arduino. (2024). "Nano 33 BLE Sense Rev2: Technical Specifications." *Arduino Documentation*. Available at: <https://docs.arduino.cc/hardware/nano-33-ble-sense-rev2/#tech-specs>. Accessed on: 1st December 2024.
10. Craig, J. J. (2005). *Introduction to Robotics: Mechanics and Control* Pearson Prentice Hall.

11. Petrov, P. L. (2023). IMU Sensor Fusion with Quaternion Interval-Based Orientation Filters Processed over Smartphone Scanned Data. *XXXII International Scientific Conference Electronics (ET)*, Sozopol, Bulgaria, 1-5. Available at: <https://doi.org/10.1109/ET59121.2023.1027976>
12. Sabatini, A. M. (2006). Quaternion-based strap-down integration method for applications of inertial sensing to gait analysis. *Medical & Biological Engineering & Computing*, 44(1-2), 101–109.
13. Noureldin, A., Karamat, T. B., & Georgy, J. (2013). *Fundamentals of Inertial Navigation, Satellite-based Positioning and their Integration*, Chapter 3: "Magnetometer-based Techniques", pp. 87-92. Springer, New York, NY.
14. Mahony, R., Hamel, T., & Pflimlin, J.-M. (2008). Nonlinear Complementary Filters on the Special Orthogonal Group. *IEEE Transactions on Automatic Control*, 53(5), 1203–1218.
15. Jankowski, M.; Szermer, M.; Zajac, P.; Amrozik, P.; Maj, C.; Nazdrowicz, J.; Jabłoński, G.; Sako9wicz, B. An Experimental Investigation of Noise Sources' Contribution in the Multi-Chip Module Open-Loop Comb-Drive Capacitive MEMS Accelerometer. *Electronics* **2024**, *13*, 2599. <https://doi.org/10.3390/electronics13132599>
16. Caballero, C.; Barbado, D.; Moreno, F.J. Human Motor Noise Assessed by Electromagnetic Sensors and Its Relationship with the Degrees of Freedom Involved in Movement Control. *Sensors* **2023**, *23*, 2256. <https://doi.org/10.3390/s23042256>
17. Pahl, A.; Rathjen, K.-U.; Dickmann, S. Intended Electromagnetic Interference with Motion Detectors. *2021 IEEE International Joint EMC/SI/PI and EMC Europe Symposium*, Raleigh, NC, USA, 2021, pp. 324-328. <https://doi.org/10.1109/EMC/SI/PI/EMCEurope52599.2021.9559187>
18. Butterworth, S. (1930). On the Theory of Filter Amplifiers. *Wireless Engineer*, 7(6), 536-541.
19. Ghosh, S., Gholap, Y., & Tallur, S. (2023). Wearable sensing module for Table Tennis stroke detection. In *Proceedings of the 2023 IEEE Applied Sensing Conference (AP-*

- SCON) (pp. 1–3). doi: <https://doi.org/10.1109/APSCON56343.2023.10101257>
20. Anik, M. A. I., Hassan, M., Mahmud, H., & Hasan, M. K. (2016). Activity recognition of a badminton game through accelerometer and gyroscope. In *Proceedings of the 2016 19th International Conference on Computer and Information Technology (ICCIT)* (pp. 213–217). doi:10.1109/ICCITECHN.2016.7860197
 21. Seong, M., Kim, G., Yeo, D., & others. (2024). MultiSenseBadminton: Wearable Sensor-Based Biomechanical Dataset for Evaluation of Badminton Performance. *Scientific Data*, 11, 343. <https://doi.org/10.1038/s41597-024-03144-z>
 22. MAXIM Integrated. (2017). MAX30102 Pulse Oximeter and Heart Rate Sensor Module. *Datasheet*. Retrieved from: <https://datasheets.maximintegrated.com/en/ds/MAX30102.pdf>
 23. SparkFun. (2024). SparkFun MAX3010x Sensor Library. GitHub Repository. Available at: https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library
 24. SparkFun Tutorials. (2024). MAX30102 Pulse Oximetry and Heart Rate Sensor Hookup Guide. Available at: <https://learn.sparkfun.com/tutorials/max30102-pulse-oximetry-and-heart-rate-sensor-hookup-guide>
 25. Arduino Documentation. "Arduino Nano 33 BLE Sense Rev2 Tutorials". Available at: <https://docs.arduino.cc/hardware/nano-33-ble-sense-rev2/#tutorials>.
 26. Arduino Documentation. "BLE Device-to-Device Communication Tutorial". Available at: <https://docs.arduino.cc/tutorials/nano-33-ble-sense-rev2/ble-device-to-device>.
 27. "Bluetooth Smart or Version 4.0+ of the Bluetooth specification". *bluetooth.com*. Archived from the original on 10 March 2017.
 28. University of Southampton, *University of Southampton Health & Safety Risk Assessment: General ECS Teaching Lab Risk Assessment*, 2024. Available at: <https://secure.ecs.soton.ac.uk/notes/ellabs> [Accessed December 1, 2024].

Appendix

A. Formulas

The Formulas used to calculate the Euler Angles used in the project.

$$\phi_{acc} = \arctan 2(a_y, a_z) \quad (1)$$

$$\theta_{acc} = \arctan \left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}} \right) \quad (2)$$

$$\psi_{acc} = \arctan 2(m_y, m_x) \quad (3)$$

$$\begin{aligned} \phi_{gyro} &= \omega_x \Delta t \\ \theta_{gyro} &= \omega_y \Delta t \\ \psi_{gyro} &= \omega_z \Delta t \end{aligned} \quad (4)$$

B. BLE UUID Table

This appendix lists the BLE services and eigenvalue UUIDs used by the device:

service/features	UUID
Service UUID	19B10000-E8F2-537E-4F6C-D104768A1214
Roll	19B10001-E8F2-537E-4F6C-D104768A1214
Pitch	19B10002-E8F2-537E-4F6C-D104768A1214
Yaw	19B10003-E8F2-537E-4F6C-D104768A1214
Heart Rate	19B10004-E8F2-537E-4F6C-D104768A1214
SpO2	19B10005-E8F2-537E-4F6C-D104768A1214

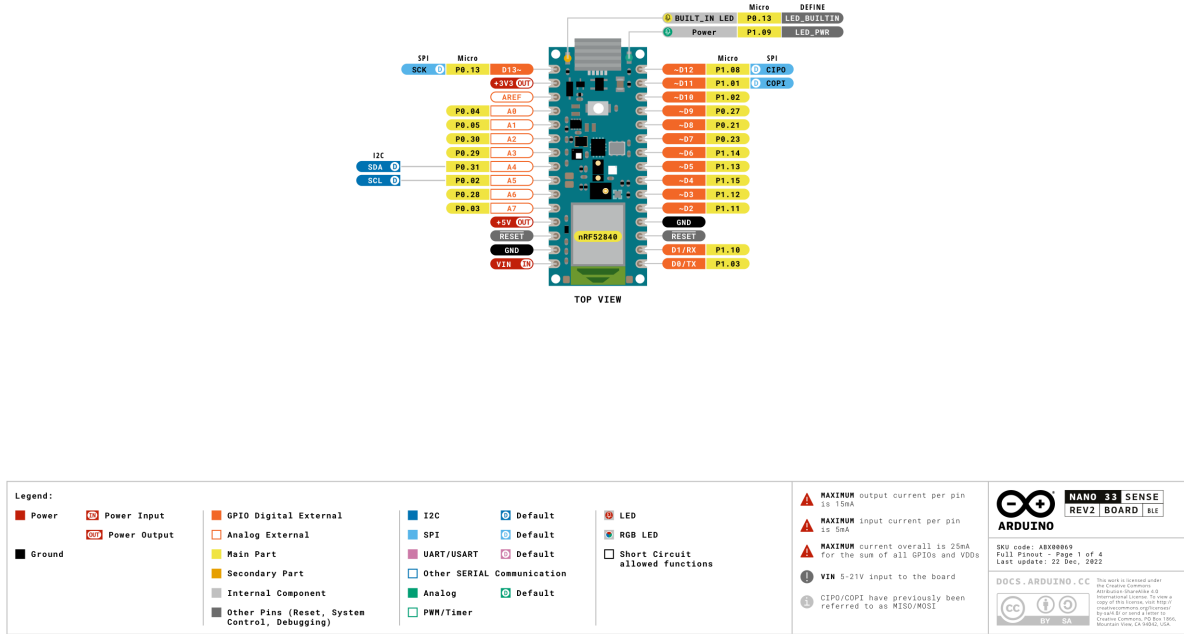


Figure 20: Pin layout of Arduino Nano 33 BLE Sense Rev2, reproduced form [9]

C. Sensor Specifications

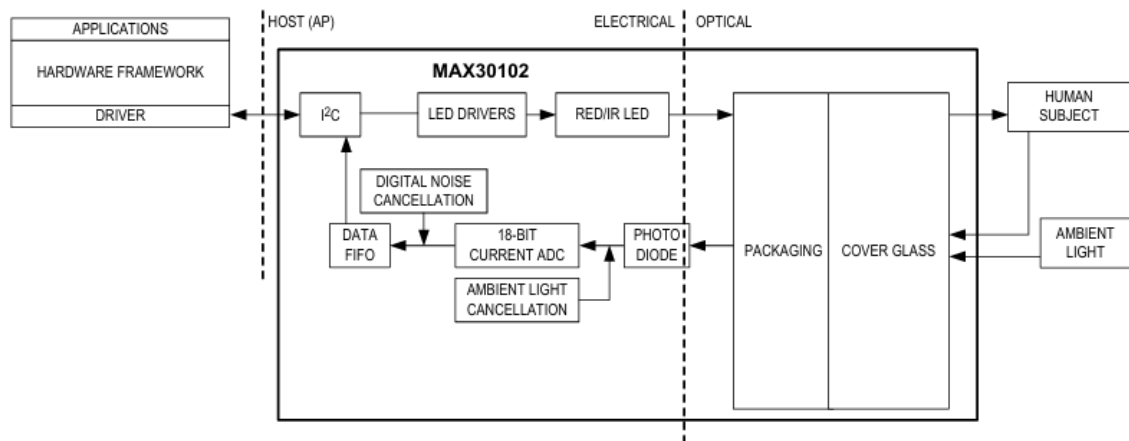


Figure 21: System Diagram of MAX30102, reproduced form [22]

D. Ethics Approval Forms

Version: 2.0

Standard ECS Operating Procedure (SOP)	B16.1003 General Bench use and access * HARDWARE PROJECTS *
---	--

Cohort: (Please Circle one only)	Y1	Y2		MSc Term Time	MSc Summer	Other (Please state)
	☒		GDP: Group Number:	BiolnspRobo: Group Number:		

Student/applciant name	<i>Yike Zhang</i>	Project Supervisor	<i>Alex Weddell</i>
Student/applciant email	<i>Y250w2@open.ac.uk</i>	Project Supervisor email	<i>aw@ecs.soton.ac.uk</i>
Student/applciant ID number	<i>33409192</i>	* Office Use Only Authorised Lab Manager	

Standard Operating Procedure(s):

B16.1005 access restrictions:	7 days per week, 08:00-23:00 (Safety Officer will advise of other closures by email)
Restricted area(s) within B16.1005	General B16.1003 users agree not use the designated High Voltage area or High Voltage equipment within B16.1007.
For awareness: Refer to the following General RCS Teaching Labs Risk Assessment Form and complete tick grid (questionnaire immediately below)	https://secure.ecs.soton.ac.uk/notes/lab/b16/B16.1003/General ECS Teaching Labs Risk Assessment.pdf

Your Project:

Please supply a working title for your project and a brief description

<i>Wearable Device for Badminton Motion detection.</i>
--

Version: 2.0

Instructions:
Tick to confirm understanding of risks and control measures:

Risk summary	I have understood this risk (tick) AND the associated control measures (tick required)	Risk summary	I have understood this risk (tick) AND the associated control measures (tick required)	Risk summary	I have understood this risk (tick) AND the associated control measures (tick required)
1) Exposure to Chemicals	☒	8) Injuries : Falling objects	☒	15) Injuries : Component overheating / explosion	☒
2) Exposure to High Voltage, current or temperatures	☒	9) Injuries : Work at height	☒	16) Injury due to faulty mains powered equipment	☒
3) Exposure to potentially hazardous waste chemicals/sharps	☒	10) Emergencies : Lone Working	☒	17) Sudden illness : Welfare/Hygiene	☒
4) Exposure to fumes from chemicals	☒	11) Injuries : Musculoskeletal / DSE	☒	18) Injuries : Flying debris	☒
5) Security of the Laboratory	☒	12) Injuries : NEW EQUIPMENT related	☒	19) Injuries : cuts and abrasions	☒
6) Injuries : Fire, burns and smoke inhalation	☒	23) Debris, Food & Drink in proximity to equipment	☒	20) Battery charging	☒
7) Injuries : Falls (on same level)	☒	14) Injuries : Soldering & rework	☒	21) Drone operation	☒
				22) COVID-19 General precautionary Measures	☒

I agree to the above Operating Procedure:

Student signature <i>Yike Zhang</i>	Date <i>21.10.24</i>
--	-------------------------

I have ensured that the above person understands the importance of this Operating Procedure:

Project supervisor signature <i>Alex Weddell</i>	Date <i>21/10/24</i>
---	-------------------------

Figure 22: Signed SOP Form

100685 - Wearable Device for Badminton Motion Detection					
Submissions Overview Submission Questionnaire Attachments History					
Status Awaiting Supervisor Review Category Faculty of Engineering and Physical Sciences (EPS)					
Amendment History Original Submission 100685 (Created 2017-10-04)					
User Uploaded Documents 1 Download (18 documents)					
Title	Version Number	Document Date	Document Type	Size	
1. 1. Ethics Application Form	1	04/12/2024	Ethics Form	44 KB	
1.1. Ethics Application Form	1	20/11/2024	Ethics Form	43 KB	
1.2. Ethics Application Form	1	20/11/2024	Ethics Form	43 KB	
2. 2. Ethics Application Form for Secondary Data Analysis	1	04/12/2024	Ethics Form	43 KB	
2.1. Ethics Application Form for Secondary Data Analysis	1	04/12/2024	Ethics Form	43 KB	
2.2. Ethics Application Form for Secondary Data Analysis	1	04/12/2024	Ethics Form	43 KB	
2.3. Ethics Application Form for Secondary Data Analysis	1	20/11/2024	Ethics Form	43 KB	
3. 3. GDPR PS v2.1 - Final Feb 24	2.1	04/12/2024	Participant Information Sheet	73 KB	
3.1. GDPR PS v2.1 - Final Feb 24	2.1	04/12/2024	Participant Information Sheet	73 KB	
3.2. GDPR PS v2.1 - Final Feb 24	2.1	04/12/2024	Participant Information Sheet	73 KB	
3.3. GDPR PS v2.1 - Final Feb 24	2.1	20/11/2024	Participant Information Sheet	73 KB	
4. 4. LMS updated Consent Form v2.1 - 20.01.24	2.1	04/12/2024	Consent Form	40 KB	
4.1. LMS updated Consent Form v2.1 - 20.01.24	2.1	20/11/2024	Consent Form	40 KB	
4.2. LMS updated Consent Form v2.1 - 20.01.24	2.1	04/12/2024	Other	27 KB	
4.3. LMS updated Consent Form v2.1 - 20.01.24	2.1	04/12/2024	Other	28 KB	

Figure 23: Ethics Approval Submissions

E. Code Examples

Hardware code structure:

```
Project Root/  
├── libraries/  
│   ├── Arduino_BMI270_BMM150/  
│   ├── ArduinoBLE/  
│   ├── Madgwick/  
│   └── MAX3010x_Sensor_Library-1.0.5/  
└── src/  
    ├── Bluetooth.cpp  
    ├── Bluetooth.h  
    ├── Filter.cpp  
    ├── Filter.h  
    ├── HeartRateSpO2.cpp  
    ├── HeartRateSpO2.h  
    └── main.ino
```

```
122 void readAndFuseIMUData() {  
123     if (IMU.accelerationAvailable()) {  
124         IMU.readAcceleration(ax, ay, az);  
125     }  
126     if (IMU.gyroscopeAvailable()) {  
127         IMU.readGyroscope(gx, gy, gz);  
128     }  
129     if (IMU.magneticFieldAvailable()) {  
130         IMU.readMagneticField(mx, my, mz);  
131     }  
132     if (!isValidIMUData(ax, ay, az, gx, gy, gz, mx, my, mz)) {  
133         Serial.println("Not valid data, loading failed!");  
134         return;  
135     }
```

Figure 25: Code example of Reading Data

```

3 // service UUID & feature UUID
4 #define SENSOR_SERVICE_UUID "19B10000-E8F2-537E-4F6C-D104768A1214"
5 #define ROLL_CHARACTERISTIC_UUID "19B10001-E8F2-537E-4F6C-D104768A1214"
6 #define PITCH_CHARACTERISTIC_UUID "19B10002-E8F2-537E-4F6C-D104768A1214"
7 #define YAW_CHARACTERISTIC_UUID "19B10003-E8F2-537E-4F6C-D104768A1214"
8 #define HEART_RATE_CHARACTERISTIC_UUID "19B10004-E8F2-537E-4F6C-D104768A1214"
9 #define SPO2_CHARACTERISTIC_UUID "19B10005-E8F2-537E-4F6C-D104768A1214"
10
11 BLEService sensorService(SENSOR_SERVICE_UUID);
12
13 BLEFloatCharacteristic rollCharacteristic(ROLL_CHARACTERISTIC_UUID, BLERead | BLENotify);
14 BLEFloatCharacteristic pitchCharacteristic(PITCH_CHARACTERISTIC_UUID, BLERead | BLENotify);
15 BLEFloatCharacteristic yawCharacteristic(YAW_CHARACTERISTIC_UUID, BLERead | BLENotify);
16 BLEFloatCharacteristic heartRateCharacteristic(HEART_RATE_CHARACTERISTIC_UUID, BLERead | BLENotify);
17 BLEFloatCharacteristic spo2Characteristic(SPO2_CHARACTERISTIC_UUID, BLERead | BLENotify);
18
19 BLEDevice central;

```

Figure 26: Code example of Establish BLE Connection

```

66 void HeartRateSpO2::calculateHeartRateAndSpO2() {
67     applyMovingAverageFilter(irBuffer, BUFFER_SIZE, 4);
68     applyMovingAverageFilter(redBuffer, BUFFER_SIZE, 4);
69
70     float irMean = calculateMean(irBuffer, BUFFER_SIZE);
71     float redMean = calculateMean(redBuffer, BUFFER_SIZE);
72
73     float irAC = 0;
74     float redAC = 0;
75     for (int i = 0; i < BUFFER_SIZE; i++) {
76         irAC += abs((float)irBuffer[i] - irMean);
77         redAC += abs((float)redBuffer[i] - redMean);
78     }
79     irAC /= BUFFER_SIZE;
80     redAC /= BUFFER_SIZE;
81
82     float R = (redAC / redMean) / (irAC / irMean);
83
84     spo2 = 110.0 - 25.0 * R;
85     spo2 = constrain(spo2, 0, 100);
86
87     int beatCount = 0;
88     for (int i = 1; i < BUFFER_SIZE - 1; i++) {
89         if (irBuffer[i] > irBuffer[i - 1] && irBuffer[i] > irBuffer[i + 1] && irBuffer[i] > irMean * 1.2) {
90             beatCount++;
91         }
92     }
93
94     heartRate = (float)beatCount * (60.0 / (BUFFER_SIZE * dt));
95 }
96

```

Figure 27: Code example of Heartrate and SpO_2 Algorithm

```

149 function peakCount = countPeaks(data)
150     if length(data) < 3
151         peakCount = 0;
152         return;
153     end
154     peakCount = sum(diff(sign(diff(data))) < 0);
155 end
156

```

Figure 28: Peak Count Algorithm

```

1 function runBleContinuousCaptureColumnsPlot()
2     clc; clear; close all;
3
4     %csvFileName = "pre_test.csv";
5
6     timestamp = datestr(now, 'yyyymmdd_HHMM');
7     csvFileName = sprintf("pre_test_%s.csv", timestamp);
8
9     fileID = fopen(csvFileName, 'w');
10    if fileID == -10
11        error('Fail to create/open file %s, please check the path and make sure file is not in use!', csvFileName);
12    end
13
14    fprintf(fileID, 'timestamp,roll,pitch,yaw,heartRate,spo2,roll_velocity,pitch_velocity,yaw_velocity,roll_acceleration,pitch_acceleration,yaw_acce
15
16    targetName = "TOGENASHI_TOGEARI";
17    disp("Scanning for BLE devices...");
18    devs = blelist;
19    disp(devs);
20
21    idx = find(strcmp(devs.Name, targetName), 1);
22    if isempty(idx)
23        fclose(fileID);
24        error("Device '%s' not found in scan results.", targetName);
25    end
26    deviceAddr = devs.Address(idx);
27    disp("Found device: " + targetName + ", Address = " + deviceAddr);
28
29    b = tryConnectAddress(deviceAddr, 3);
30    if isempty(b) || ~isvalid(b)
31        fclose(fileID);
32        error("Failed to connect to %s (%s).", targetName, deviceAddr);
33    end
34    disp("Connected to " + targetName + " (" + deviceAddr + ")");
35

```

Figure 29: The function for Connecting and Data Storage

```

115 @app.route('/sensor_data')
116 def get_sensor_data():
117     loop = asyncio.new_event_loop()
118     asyncio.set_event_loop(loop)
119     loop.run_until_complete(read_sensor_data())
120
121     roll = sensor_values.get("roll", 0)
122     pitch = sensor_values.get("pitch", 0)
123     yaw = sensor_values.get("yaw", 0)
124
125     dt = 0.01
126
127     roll_velocity = (roll - get_sensor_data.prev_roll) / dt
128     pitch_velocity = (pitch - get_sensor_data.prev_pitch) / dt
129     yaw_velocity = (yaw - get_sensor_data.prev_yaw) / dt
130
131     roll_acceleration = (roll_velocity - get_sensor_data.prev_roll_v) / dt
132     pitch_acceleration = (pitch_velocity - get_sensor_data.prev_pitch_v) / dt
133     yaw_acceleration = (yaw_velocity - get_sensor_data.prev_yaw_v) / dt
134
135     get_sensor_data.prev_roll = roll
136     get_sensor_data.prev_pitch = pitch
137     get_sensor_data.prev_yaw = yaw
138     get_sensor_data.prev_roll_v = roll_velocity
139     get_sensor_data.prev_pitch_v = pitch_velocity
140     get_sensor_data.prev_yaw_v = yaw_velocity
141
142     get_sensor_data.roll_velocity_buffer.append(roll_velocity)
143     if len(get_sensor_data.roll_velocity_buffer) > 10:
144         get_sensor_data.roll_velocity_buffer.pop(0)
145
146     peak_count = 0
147     if len(get_sensor_data.roll_velocity_buffer) >= 3:
148         diff_sign = np.sign(np.diff(get_sensor_data.roll_velocity_buffer))
149         peak_count = int(np.sum(np.diff(diff_sign) < 0))
150

```

Figure 30: Code example of Data Processing Progress in Pyhton

```

7  <style>
8    body {
9      font-family: 'Segoe UI', Tahoma, sans-serif;
10     background-color: #f6f8fa;
11     color: #2c3e50;
12     max-width: 800px;
13     margin: auto;
14     padding: 2rem;
15   }
16
17   h1, h2, h3 {
18     color: #1a73e8;
19     margin-bottom: 0.5rem;
20   }
21
22   .card {
23     background-color: #ffffff;
24     border-radius: 12px;
25     box-shadow: 0 2px 10px rgba(0,0,0,0.05);
26     padding: 1.5rem;
27     margin-bottom: 2rem;
28   }
29
30   .data-pair {
31     margin: 0.5rem 0;
32     font-size: 1.05rem;
33   }
34
35   span.value {
36     font-weight: bold;
37     color: #0f9d58;
38   }
39
40   #feedback {
41     margin-top: 1rem;
42     padding: 1rem;
43     background-color: #e3f2fd;
44     border-left: 4px solid #1a73e8;
45     border-radius: 6px;
46   }
47 </style>

```

Figure 31: Code example of UI

```
151     features = {
152         "roll_velocity": roll_velocity,
153         "pitch_velocity": pitch_velocity,
154         "yaw_velocity": yaw_velocity,
155         "roll_acceleration": roll_acceleration,
156         "pitch_acceleration": pitch_acceleration,
157         "yaw_acceleration": yaw_acceleration,
158         "peak_count": peak_count,
159         "roll": roll,
160         "pitch": pitch,
161         "yaw": yaw,
162         "roll_velocity_abs": abs(roll_velocity),
163         "pitch_velocity_abs": abs(pitch_velocity),
164         "yaw_velocity_abs": abs(yaw_velocity),
165         "roll_acceleration_abs": abs(roll_acceleration),
166         "pitch_acceleration_abs": abs(pitch_acceleration),
167         "yaw_acceleration_abs": abs(yaw_acceleration)
168     }
169
```

Figure 32: Code example of Extended 16 Features